Taaltheorie en Taalverwerking

BSc KI / Artificial Intelligence

2026 – Lecture 1

Katrin Schulz

k.schulz@uva.nl

Institute for Logic, Language, and Computation

University of Amsterdam

Welcome to Taaltheorie en Taalverwerking
Language theory and language processing

- What is this course about?
  - Modeling and processing languages, both computer languages and human languages
- What will we study in this course?
  - Formal languages and automata
  - Algorithms to automatically extract the structure of sentences
  - Algorithms and formalisms to analyze the meaning of sentences
  - Rule-based (symbolic) approaches as well as data-driven (statistical or machine learning) approaches

Why does it matter?

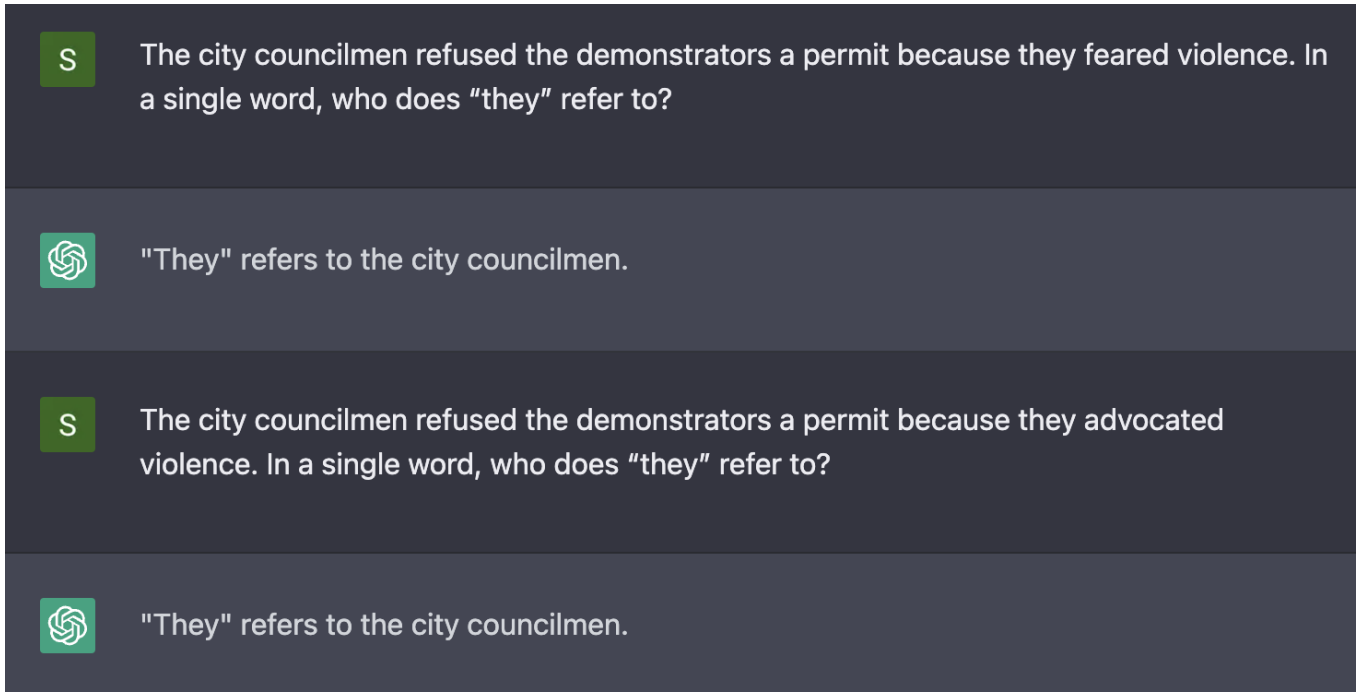
Natural language is one of the hallmarks of *human* intelligence, so it is of utmost interest also for *artificial* intelligence!

- Interaction with computers and phones
- Automatically extracting information from text (news, government reports, social media)
- Many applications: next-word-prediction on your phone, automatic translation, finding relevant hits for a search, etc.
- Current state-of-the-art: generative, conversational systems powered by deep learning: `ChatGPT`, `GPT-4`, etc.

How far have we come?

Haven't we already solved this problem?

Consider this example (`ChatGPT`, generated on March 17, 2023):



Levels of language processing

Language seems easy to us, but it is quite difficult for computers.

What kind of knowledge do we need to be able to *process/understand/react to* (written) language?

- syntax
  Describes the structural properties of the language.
- semantics
  Represents the meaning of words and sentences.
- pragmatics
  Relation between the meaning of a sentence and the context (world knowledge, speaker and listener intentions, conventions).

Types of languages

When studying language, we distinguish between *formal* and *natural* languages.

- Natural (human) languages: Dutch, English, Italian, Japanese, Swahili, Hindi, ...
- Formal (computer) languages: Java, Prolog, Python, LaTeX, HTML, SQL, ...

Common theoretical core

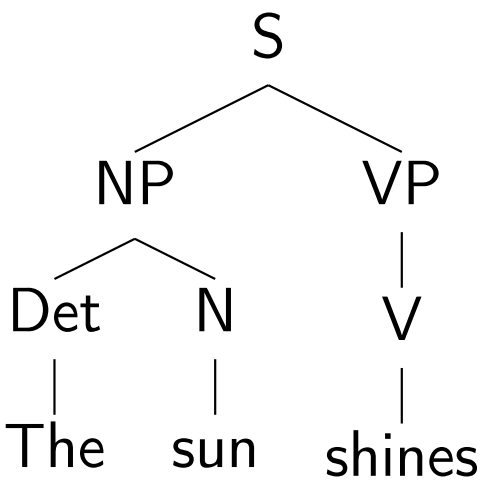
In this course, you will learn certain theoretical concepts that can be applied to each of the above domains:

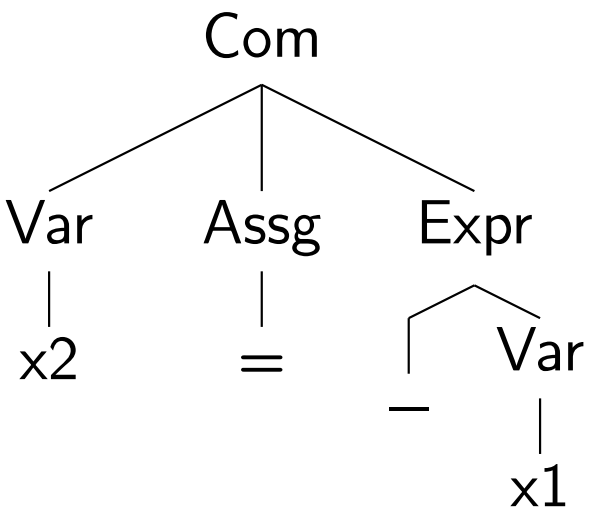
- regular languages
- finite state machines
- context-free languages, syntax trees
- types, compositional semantics

Many of the underlying theory is very fundamental, and can be applied in very diverse contexts!

Syntax trees: a central concept

Syntax : Describes the structural properties of the language. Natural language is much more complicated than the formal languages used for logic and computer programs.





Natural language semantics

Consider the sentence:

Every student has access to a computer

The 'meaning' of this can be expressed as a logical formula:

$$\forall x.(student(x) \Rightarrow \exists y.(computer(y) \wedge hasAccessTo(x, y)))$$

Or perhaps:

$$\exists y.(computer(y) \wedge \forall x.(student(x) \Rightarrow hasAccessTo(x, y)))$$

Problem: How can (either of) these formulae be mechanically generated from a syntax tree for the original sentence?

Comparison between Formal and Natural Languages

Formal (FLs) and Natural Languages (NLs) are similar in some ways, but there are also important differences.

- FLs can be pinned down by a precise definition. NLs are fluid, fuzzy at the edges, and constantly evolving.
  - Metaphorical use of words ("shining eyes")
  - Word meaning changes over time ("girl")
  - Language itself can change (Old English: "Dunor cymd of haetan & of waetan")
- NLs are riddled with ambiguity at all levels, FLs are usually not.
  - if a then b else c
  - 5 + 6 * 7

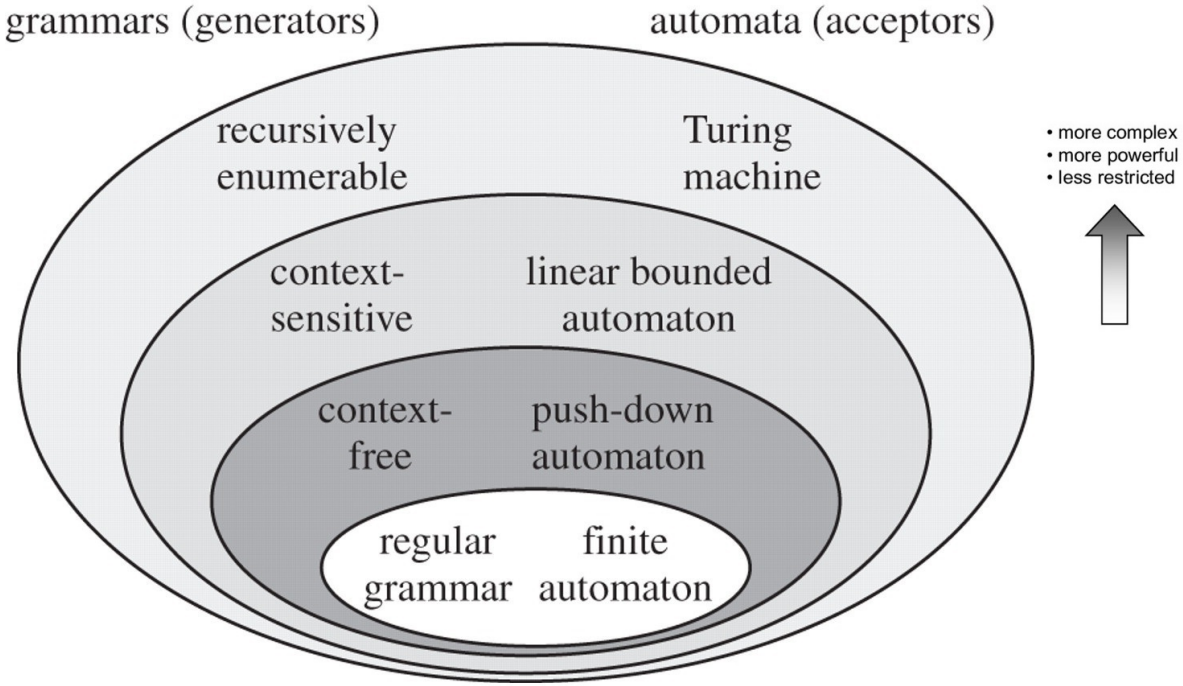
Ambiguity in natural language

There are many different types of ambiguities

- Phonological ambiguity: e.g. 'an ice lolly' vs 'a nice lolly'
- Lexical ambiguity: e.g. 'bass' has at least two meanings: fish, and musical instrument
- Syntactic ambiguity: e.g. two possible syntax trees for 'complaints about referees multiplying'
- Semantic ambiguity: e.g. 'Four women carry a piano' vs. 'Four women carry a chair'

Levels of language complexity

Languages can be more or less 'complex' (harder to describe, harder to process)



We can classify languages on a scale of complexity known as the 'Chomsky Hierarchy' by Prof. Noam Chomsky (16.1 in the handbook)

Some info about the course

The TTTV 2026 team:

- Coordinator: **dr. Sandro Pezzelle**
- Lectures: **dr. David Graus, dr. Katrin Schulz**
- TAs: **Lucas Groot, Jan Hutter, Thien An Pham, Stef de Wildt, Delaram Zakar, Frank Wildenburg** (first point of contact)
- TA coordinator: **Frank Wildenburg** (contact him for anything your TA cannot solve)

Some info about the course

Sandro Pezzelle (the coordinator):

- Assistant Professor at the Institute for Logic, Language and Computation at the University of Amsterdam
- Working in the domain of Computational linguistics (CL) / Natural Language Processing (NLP). Some research lines:
  - Computational semantics and pragmatics: how NLP models represent words, phrases, and sentences (in context)
  - Language and vision: reasoning, linguistic, and communicative abilities of visually-grounded models
  - Model interpretability: how and where large language models store their knowledge and skills
- Sandro's personal page: `https://sandropezzelle.github.io/`

Some info about the course

Katrin Schulz (one of the lecturers):

- Associate Professor at the Institute for Logic, Language and Computation at the University of Amsterdam
- Working on questions of language and cognition using tools from logic and Artificial Intelligence.
- Study how we make sense of the world, using language as "window to the mind". Some research lines:
  - Models of human (causal) reasoning
  - Models of world knowledge: how do we categorize and generalise our sensory input
  - Bias and stereotypes

Some info about the course

Practicalities:

- Weekly structure
  - 2 hoorcolleges (lectures)
  - 1 werkcollege + 1 laptopcollege
- Information
  - Course manual: syllabus, contents, rules...
  - Canvas: materials, readings, assignments, announcements ...
  - DataNose: schedule, groups, and classrooms
- Grades
  - 2 exams: one on April 23 (40%), one on May 29 (40%)
  - 5 weekly assignments done in pairs: pen-and-paper exercises + labs (20%)
  - To pass the course, it is necessary (not sufficient) to **obtain at least a 5 for both exams together**.

Learning objectives

The student...

- is able to analyze the structure and the meaning of natural language expressions
- can use the basic concepts of computational linguistics to define an NLP problem formally
- is able to apply computational processing to parse syntax, semantics, and morphology
- can describe the computational challenges of this parsing process
- is able to use common NLP libraries from a general programming language

Questions?

Let's get started

This week:

- Formal language theory
- Regular languages

Formal languages: strings and alphabets

- A formal language is a set of strings.
- Each string is composed of symbols from a set called an alphabet (or a vocabulary).

Examples of alphabets

- English letters: $\Sigma = \{a, b, c \ldots z\}$
- Decimal digits: $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- Programming language 'tokens' : $\Sigma = \{if, while, x, ==\}$
- Words in (some fragment of) a natural language: $\Sigma = \{the, an, a, dog, cat, sleeps\}$

Formal language: strings and alphabets

- A formal language is a set of strings.
- Each string is composed of symbols from a set called an alphabet (or a vocabulary).

Examples of strings over alphabets

- Let $\Sigma_1 = \{0, 1\}$ be an alphabet.Then 01101, 000001, 1101 are strings over Σ_1 — in fact, all binary numbers are strings over Σ_1
- Let $\Sigma_2 = \{a, b, c, d, e, f, g\}$ be an alphabet. Then *bee*, *dad*, *cabbage*, and *face* are strings over Σ_2, as are *fffff* and *agagag*.
- Let $\Sigma_3 = \{ba, ca, fa, ce, fe, ge\}$ be an alphabet. Then *face* is a string over Σ_3 but *bee*, *dad* or *cabbage* are <u>not</u>.
- Let $\Sigma_4 = \{\spadesuit, \triangle, \clubsuit\}$ be an alphabet. Then $\spadesuit\spadesuit$ and $\clubsuit\triangle\clubsuit$ are strings over Σ_4.

Strings and substrings

The length of a string is the number of token symbols from the alphabet it contains.

Examples

- the length of *face* over $\Sigma_2 = \{a, b, c, d, e, f, g\}$ is 4
- the length of *face* over $\Sigma_3 = \{ba, ca, fa, ce, fe, ge\}$ is 2

The string of length 0 is called the empty string, denoted ϵ (epsilon).

Given a string s, a substring of s is a string formed by taking contiguous symbols of s in the order in which they occur in s.

- An initial substring is called a prefix
- and a final substring, a suffix.

Examples

Let *unthinkable* be a string over $\Sigma = \{a, b, c \ldots x, y, z\}$
Then, ϵ, *un*, *unth*, *unthinkable* are prefixes, while ϵ, *e*, *able*, *thinkable*, and *unthinkable* are suffixes. Other substrings include *nthi*, *inka*, *bl*.

Formal languages

Recall that a formal language is just a set of strings.

- Σ^* denotes the set of all strings over an alphabet Σ
  - $\rightarrow$ Note that Σ^* is always infinite, regardless of the number of symbols Σ contains.

We may now define a formal language L over an alphabet Σ as any subset of Σ^*:

$$L \subseteq \Sigma^*$$

Formal languages

- A formal language L over an alphabet Σ is any subset of Σ^*

$$L \subseteq \Sigma^*$$

Examples of formal languages

Let $\Sigma = \{a, b, c \ldots x, y, z\}$. Then Σ^* is the set of strings over the Latin alphabet. The following subsets of Σ^* are possible formal languages:

- the empty set
- the set $\{one, two, three, four, five, six, seven, eight, nine, ten\}$
- the set of strings whose length is less than 9 symbols
- the set of all English words
- **the set of strings consisting of consonants only**
- **the set of strings containing at least one vowel and one consonant**

Finite vs Infinite languages

- A **finite** language is any set $\mathscr{L}$ of strings, of finite cardinality, $|\mathscr{L}| < \infty$
- An **infinite** language is any set $\mathscr{L}$ of strings, of infinite ($\aleph_0$) cardinality $|\mathscr{L}| = \infty$

Ways to define a formal language

How can we formally define or specify the language(s) we are interested in?

- Given an alphabet Σ and the infinite set Σ^* of formal languages it can give rise to, how can we select a particular formal language?

  For instance:

  - Can we define the set of strings of letters that constitute proper English words?
  - Can we define the set of strings of words that count as well-formed sentences of Dutch?

Ways to define a formal language

How can we formally **define or specify** the language(s) we are interested in?

1. Direct mathematical definition: $\Sigma = \{a, b, c\}$

$$L_1 = \{aa, bb, abc, abcc\}$$
$$L_2 = \{a^n b^n | n > 0\}$$

2. Formalisms (formal expressions and grammars): sets of rules
   - *Declarative method*

3. Automata: computational devices for computing languages
   - Specify some machine for testing whether a string is legal
   - *Procedural method*

Formalisms and automata

- Formalisms and automata allow us to distinguish a formal language of interest (a set of strings) from other possible languages over a given alphabet.
  - They capture the patterns that characterize a language.
  - As such, they act as a definition of the language they capture.
- From an abstract point of view, a natural language – like Dutch or English – is a set of strings of sounds/letters, words, etc.
- Therefore, **formalisms and automata can help us to model aspects of natural languages**.

This week:
One formalism to define formal languages, regular expressions, and one type of automaton, finite-state automata.

Formalisms

Regular expressions

Regular expressions are a formal notation for characterizing sets of strings that follow a fairly simple regular pattern.

We can construct regular expressions over an alphabet Σ as follows:

	Regular expression	**Languages**
empty string:	$\emptyset$	$\{\}$
any symbol $a \in \Sigma$:	a	$\{a\}$
If a, b regexps, so are:		
concatenation:	$a \cdot b$	$\{ab\}$
disjunction (or union):	$(a\|b)$	$\{a, b\}$
Kleene star (or closure):	a^*	$\{\epsilon, a, aa, aaa, aaaaa, \ldots\}$

- we often ignore the dot in concatenation ($a \cdot b$) and simply write ab
- disjunction (or union) may be written as $a|b$, $a + b$ or $a \cup b$
- a^+ is the set of a-strings with at least one a; it is an abbreviation of a^*a or aa^*
- the notation Σ^* can be seen as abbreviating $(a|b|...)^*$

Regular expressions: examples

What kind of strings would the language defined by each of the following regular expressions contain?

Let $\Sigma = \{a, b, c, d ... x, y, z\}$

Regular expression	
$me(o)^*w$	$\{mew, meow, meoow, meooow, meoooooooow, \ldots\}$
$ba(a)^+$	$\{baa, baaaaaa, baaaaaaaaa, \ldots\}$
$(ab)^*c$	$\{c, abc, ababc, abababc, \ldots\}$
$(a^*\|b^*)c$	$\{c, ac, bc, aac, bbc, aaac, bbbc \ldots\}$

Regular expressions: Programming language notation

Many programming languages such as Perl, Python, Java, or Unix tools like `grep` include ways to specify regular expressions.

	Perl notation	**underlying regular expression**
ranges:	`[a-z]`	$(a\|b\|c\|d\|\ldots\|z)$
optionality:	`colo(u)?r`	$colo(u\|\epsilon)r$
digits:	`\d`	$(0\|1\|2\|3\|4\|5\|6\|7\|8\|9)$
negation:	`[^0-6]`	$(7\|8\|9)$

There are many other options, such as upper-/lower-case, white spaces, etc.

These operators are "syntactic sugar": all regular expressions can be constructed with the basic operations we have seen earlier: *concatenation, disjunction, and Kleene star, plus the empty string.*

This is very useful! Regexes are used all over the place for string search. This won't be covered in class – see the book and practice in the *werkcolleges*.

Let's play Wordle!

Go to
`https://www.nytimes.com/games/wordle/index.html`
and (try to) guess the word of the day in 5 minutes!

Wordle: How can we solve it?

1 Go to wooclap.com

2 Enter the event code in the top banner

Event code
XAYUMJ

Enable answers by SMS

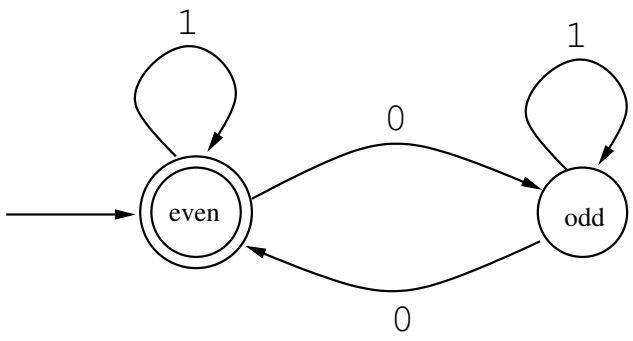
Regular expressions and automata

- A regular expression allows us to characterize a formal language declaratively, i.e., using a **set of rules**
- We can characterize the same language procedurally using an *automaton* (**machine**) that specifies how the language is computed.

Automata

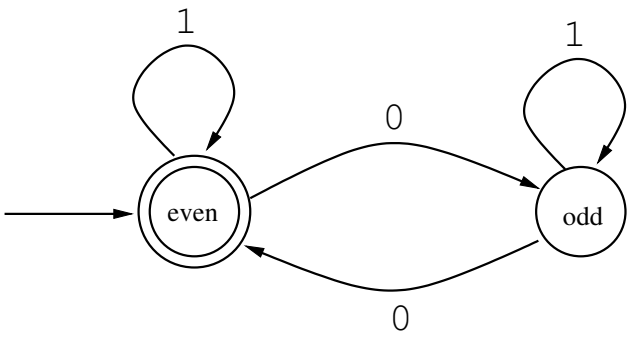
A simple FSA

A *finite state automaton* (FSA) is a machine that, at any point in time, can be in one of a *finite* number of states (circles). Its *transitions* (arrows) describe how to go from state to state



- This is an example of a finite state automaton over $\Sigma = \{0, 1\}$.
- The state marked with the in-arrow is the starting state
- At any moment, the machine is in one of 2 states. From any state, each symbol in Σ determines a 'destination' state we can move to.
- Certain states (with double circles) are designated as accepting.
- The machine defines a language $L \subseteq \Sigma^*$

Question



- For the finite state machine shown here, which of the following strings are legal (i.e. accepted)?

1. ϵ
2. 11
3. 1010
4. 1101
5. 01101

FSA: exercise

1. Go to wooclap.com
2. Enter the event code in the top banner

Event code
XAYUMJ

Enable answers by SMS

Get to the harbor! (try at home)

Get to Harbour

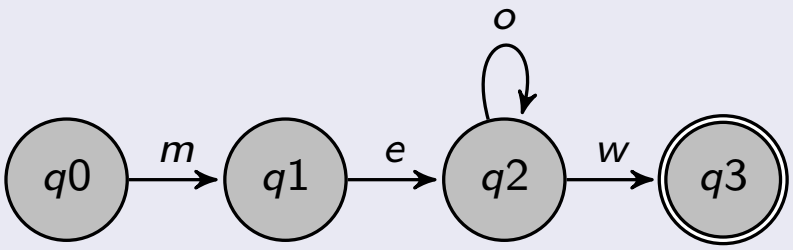
Find the sequence of trains to get from the Airport to **Harbour** (final state)

For example, if you took the A-train, then the B-train, then the A-train, type in ABA.

Regular expressions and automata: example

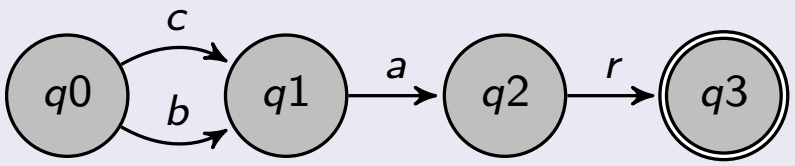
regular expression: meo^*w

language: $\{mew, meow, meoow, meooow...\}$



regular expression: $(b|c)ar$

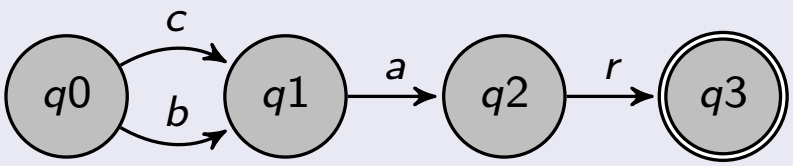
language: $\{bar, car\}$



Finite-state automata: formal definition

We can formally specify an FSA by the following 5 parameters:

- Σ: an input alphabet
- Q: a finite set of states
- $q_0 \in Q$: the start state
- F: a set of final or accepting states ($F \subseteq Q$)
- δ: a transition function between states that maps pairs of states and input symbols $\langle q, i \rangle$ to a new state q' ($Q \times \Sigma \rightarrow Q$).

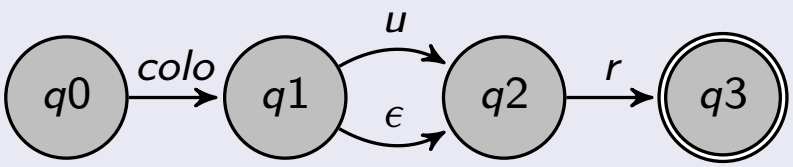


$Q = \{q_0, q_1, q_2, q_3\}$
$\Sigma = \{a, b, c, r\}$ (or a superset)
start state: q_0
$F = \{q_3\}$
$\delta = \{(\langle q_0, c\rangle, q_1), (\langle q_0, b\rangle, q_1),$
$(\langle q_1, a\rangle, q_2)\ ,\ (\langle q_2, r\rangle, q_3)\}$

Finite-state automata with ϵ-transitions

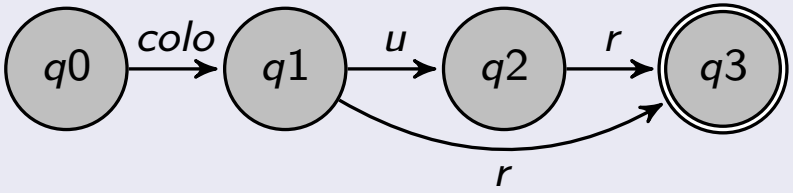
We may extend the transition function δ with the empty string symbol ϵ, so that $\delta = Q \times \Sigma \cup \{\epsilon\} \rightarrow Q$

regular expression: $colo(u|\epsilon)r$



...but note that every FSA with ϵ-transitions is equivalent to one without them:

regular expression: $colo(u|\epsilon)r$



(more on this when we discuss non-deterministic FSAs)

Regular expressions and FSAs

Regular expressions and FSAs

Regular expressions and FSAs capture the same class of languages: any language that is denoted by some regular expression can be computed by an FSA, and *vice-versa* (with some exceptions: we will see them later).

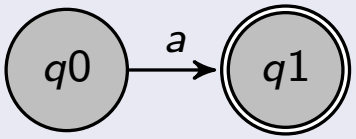
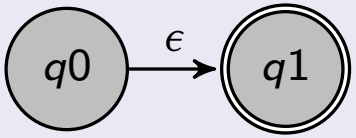
How can we build an FSA from *any* regular expression? Recall the operations that we have for reg exp:

	Reg. exp.	**Languages**
any symbol $a \in \Sigma$:	a	$\{a\}$
empty string:	ϵ	$\{\epsilon\}$
	If a and b are reg exp, so are:	
concatenation:	ab	$\{ab\}$
disjunction (or union):	$(ab\|ba)$	$\{ab, ba\}$
Kleene star (or closure):	a^*	$\{\epsilon, a, aa, aaa, aaaaa, \ldots\}$

From reg exp to FSA

Strategy:

- Base case: build an automaton for simple expressions (a, ϵ)
- Inductive step: show how to reproduce each of the operations on regular expressions with an automaton

Regular expression	Corresponding FSAs
a	
ϵ	

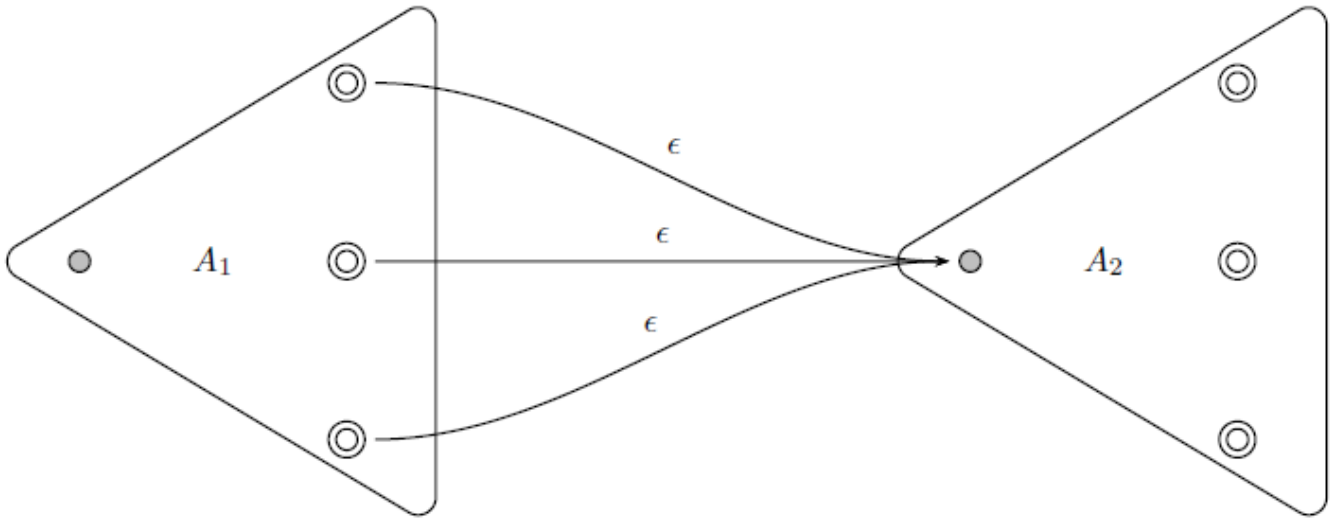
From reg exp to FSA: concatenation

Regular expression:

- If a and b are regular expressions, so is ab.

Corresponding FSA:

- Build FSA for a (A_1), build FSA for b (A_2)
- Link all final states in A_1 to the initial state of A_2 with ϵ-transitions.



This gives us a general procedure regardless of the properties of A_1 and A_2. We may be able to simplify the ϵ-transitions at the end.

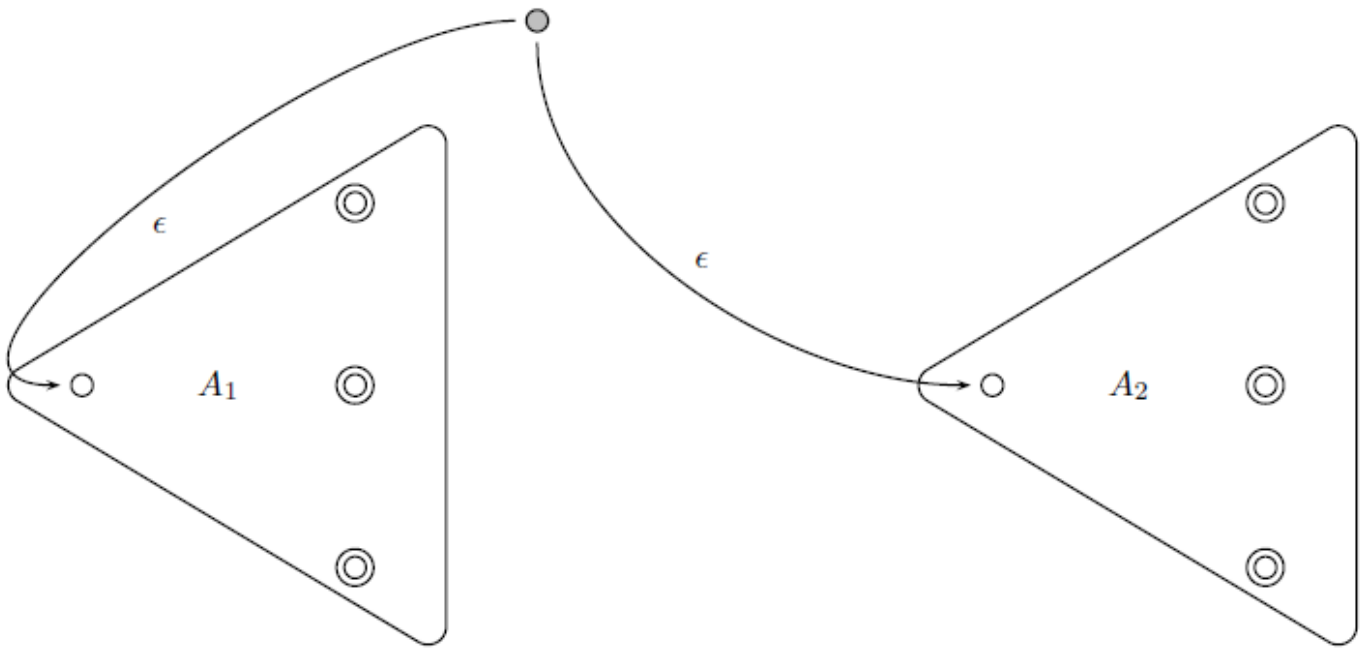
From reg exp to FSA: disjunction

Regular expression:

- If a and b are regular expressions, so is $(a|b)$.

Corresponding FSA:

- Build FSA for a (A_1), build FSA for b (A_2).
- Add a new initial state and link it to A_1 's initial state and to A_2's initial state with ϵ-transitions.



Again: this gives us a general procedure regardless of the properties of A_1 and A_2. We may be able to simplify the ϵ-transitions at the end.

From reg exp to FSA: Kleene * or closure

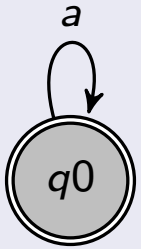
Regular expression:

- a^*: the concatenation of a with itself any number of times.

Corresponding FSA:

- Build FSA for a (A).
- Add a new final state and link all of A's final states to it with ϵ-transitions.
- Link A's initial state to the new final state with an ϵ-transition in both directions.

In the simplest case (a), after simplification, this amounts to:



Summing Up (for now)

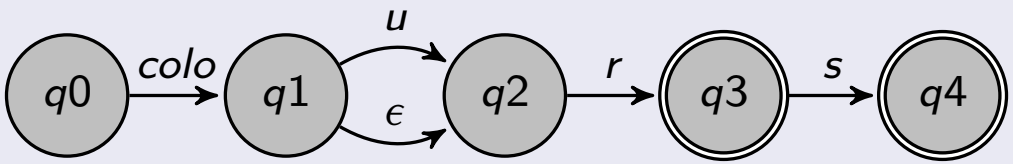
- Engaging in linguistic behavior requires different kinds of knowledge about language: syntax, semantics, pragmatics...
- We started to look into how to formally characterize the structural properties of languages using tools from formal language theory.
  - A formal language is a set of strings built up from a set of symbols, called the alphabet.
  - Regular expressions are a formalism for characterizing languages that follow simple regular patterns.
  - Many programming languages include regex functions and notation
  - Regular expressions can be characterized procedurally by means of finite state automata.
  - Regexes and FSAs capture the same class of languages.

To do...

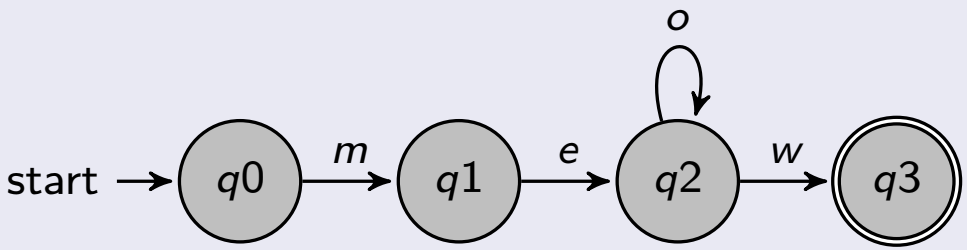
- Read the Course Manual
- Familiarize yourself with the Canvas site
- Do the reading: 3 chapters this week
- Work on the homework exercises in the *werkcolleges*
  - Pen-and-paper exercises: Submit as pdf generated by LaTeX
  - Lab exercises: Submit as iPython notebook
  - Deadline (both): Wednesday, April 8, 7pm Amsterdam time

Finite-state automata: some remarks

- There is only 1 start state...
- ...but there can be several final states.
- The start state and the final state may be the same.
- Every time we traverse the FSA from the start state to one of the final states, we have a string of the formal language (a *legal* string).
- In case a final state has an outgoing arrow, we may continue as long as we can reach a final state later on.



FSAs as generators and recognizers



FSAs can be used to generate strings:

- Each valid computation (from the start state to some final state) generates one of the possible strings of a language.

And they can also be used to recognize strings:

- Does a string belong to the language defined by the automaton?
- If the automaton consumes all input symbols in a string and reaches a final state, the string is accepted; else, it is rejected.

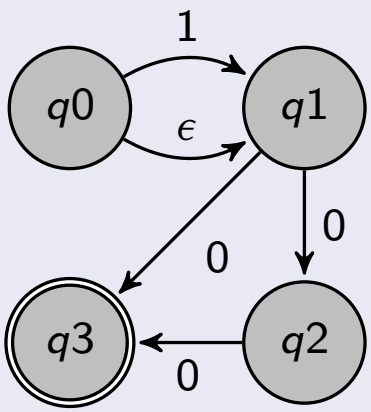
Non-deterministic FSAs (NFSA)

Deterministic FSAs

- Given a state and an input: the behavior of DFSA is fully deterministic.
  - For every state $q \in Q$ and symbol $a \in \Sigma$ there is exactly one state q' for which there exists a transition $q \xrightarrow{a} q'$ in δ.

Non-deterministic FSAs FSA is Non-deterministic if, for any state q and an input a, we can transition to more than one state.

Non-deterministic FSAa (NFSAs)



start: $q0$

$F = \{q3\}$

$$\delta = \{ \ (\langle q_0, 1\rangle, q_1), \\ (\langle q_0, \epsilon\rangle, q_1), \\ (\langle q_1, 0\rangle, q_2), \\ (\langle q_1, 0\rangle, q_3), \\ (\langle q_2, 0\rangle, q_3) \ \}$$

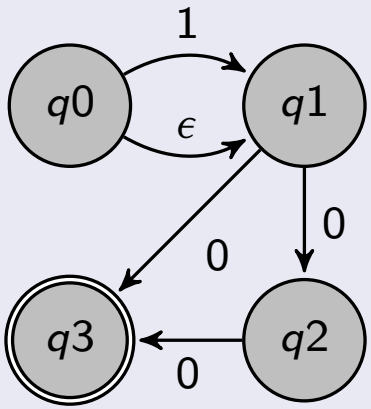
Recognizer: If we can transition to more than one state without conflicting with the input string, the FSA is non-deterministic.

Sources of non-determinism:

- states with more than one outgoing arrow with the same label (leading to different states)
- states with more than one outgoing arrow where some arrow is an ϵ-transition

Equivalence of NFSAs and DFSAs

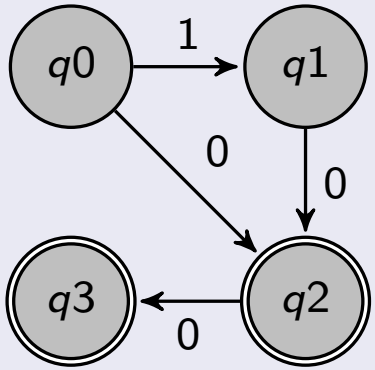
- For every NFSA, there is an equivalent DFSA – in the sense that both accept exactly the same set of strings (same language).
- Simplifying somewhat, to build a DFSA from an NFSA we need to create a deterministic path for each possible path in the NFSA.



start: $q0$

$F = \{q3\}$

$\delta = \{ (\langle q_0, 1\rangle, q_1),$
$(\langle q_0, \epsilon\rangle, q_1),$
$(\langle q_1, 0\rangle, q_2),$
$(\langle q_1, 0\rangle, q_3),$
$(\langle q_2, 0\rangle, q_3) \}$



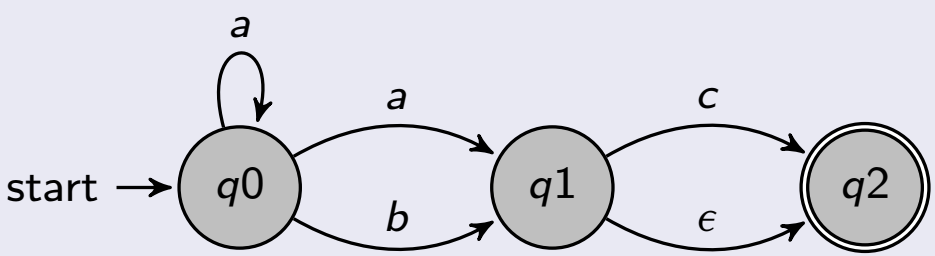
start: $q0$

$F = \{q2, q3\}$

$\delta = \{ (\langle q_0, 1\rangle, q_1),$
$(\langle q_0, 0\rangle, q_2),$
$(\langle q_1, 0\rangle, q_2),$
$(\langle q_2, 0\rangle, q_3) \}$

The recognition behavior of a DFSA is fully determined by the state it is in and the symbol it is processing.

Exercise



- What is the language generated by this FSA?
- Is it deterministic or non-deterministic?
- Why?
- Give an equivalent DFSA / FSA.

NFSAs are not more powerful than FSAs, but they may be simpler.

NFSAs as string recognizers

An NFSA accepts a string if there is at least one path (one way of consuming the input string) that leads to an accepting state.
⇒ We don't want it to stop at the first 'fail'.

String recognition with an NFSA can thus be seen as a search problem: the problem of finding an accepting path.

⇒ Recall the search techniques you have seen in the Prolog course.

The search space may be explored with two different strategies: depth-first and breadth-first.

To do...

- Read the Course Manual
- Familiarize yourself with the Canvas site
- Do the reading: 3 chapters this week
- Work on the homework exercises in the *werkcolleges*
  - Pen-and-paper exercises: Submit as pdf generated by LaTeX
  - Lab exercises: Submit as iPython notebook
  - Deadline (both): Monday, April 7, 7pm Amsterdam time